Client activity calculation

Study guide 4: activity/services.py, trading days, money flow and the manager summary

What this guide covers

`activity/services.py` (65 lines): the client activity report. For every client in an uploaded list: number of bills, distinct trading days, deposits, net withdrawals, net flow and traded or not, plus a summary per Manager and Assigned To when those columns were uploaded.

How to use it

- Read Part 1's contract carefully: this file trusts its caller to send one month of ledger rows.
- Work the example in Part 2 by hand, then rebuild the file in Part 3.
- Run the checkpoints in Part 5; they were extracted from this guide and pass against the project.

Part 1. Think before you write code

The job

activity/services.py
Given one month of ledger rows and a client list, describe each client's activity and, when possible, summarise it per manager and agent.

The contract, including what the caller must do

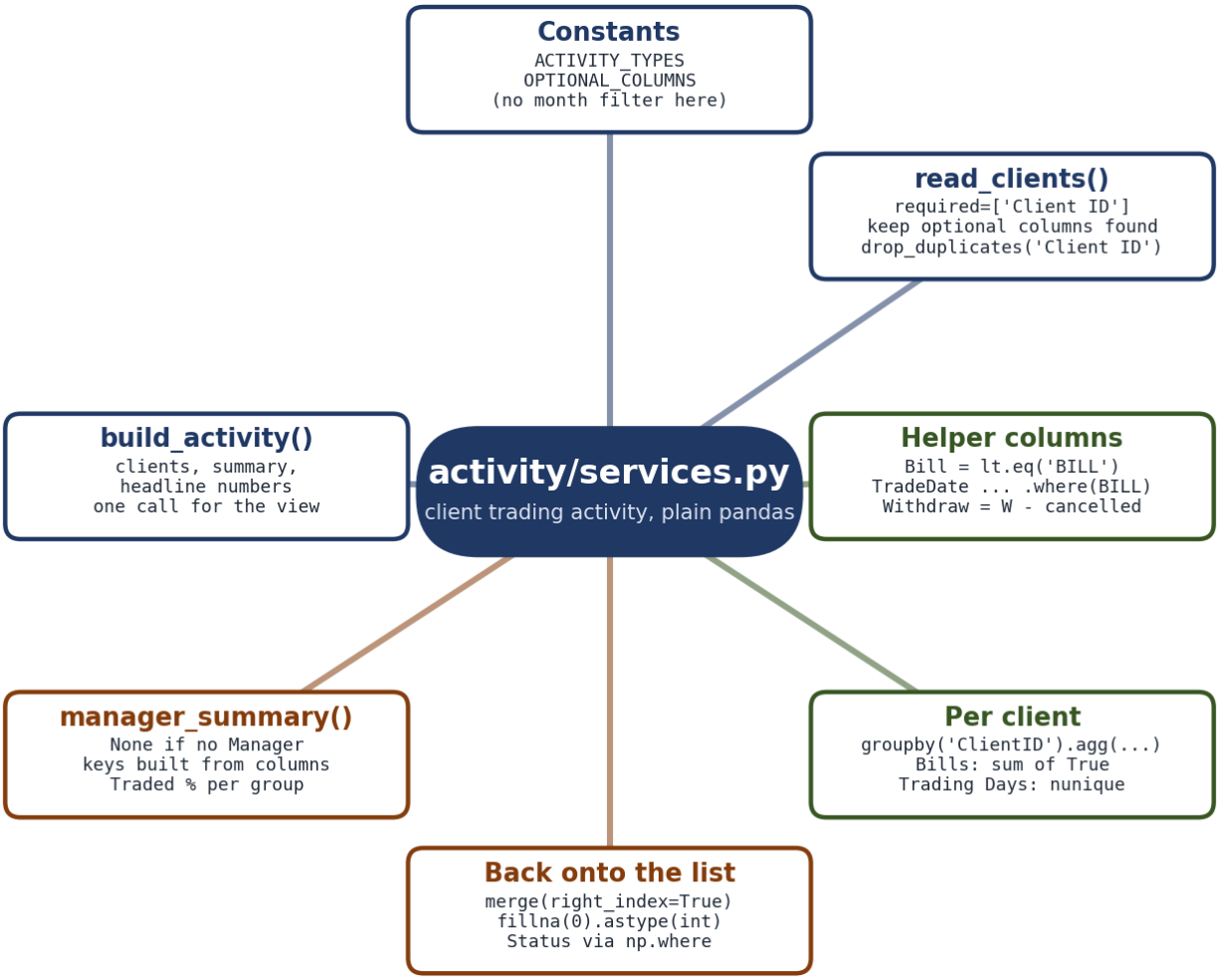
```
def build_activity(led, clients) -> dict:
    """led: ledger rows for ONE month, already limited to these clients and ACTIVITY_TYPES."""
    return {"clients": <DataFrame>, "summary": <DataFrame or None>,
            "headline": {"clients": int, "traded": int, "trading_days": int, "net_flow": float}}
```

A design choice to notice
Unlike `commission.services.client_pnl`, nothing here filters by date. The view passes `get_ledger_slice(start, end, clients=..., types=ACTIVITY_TYPES)`, so only that month arrives. That keeps this file short, but the month rule now lives in the caller. If you call `build_activity` from a notebook with the full ledger, you get all-time activity. Part 5 has an exercise to make the contract safer.

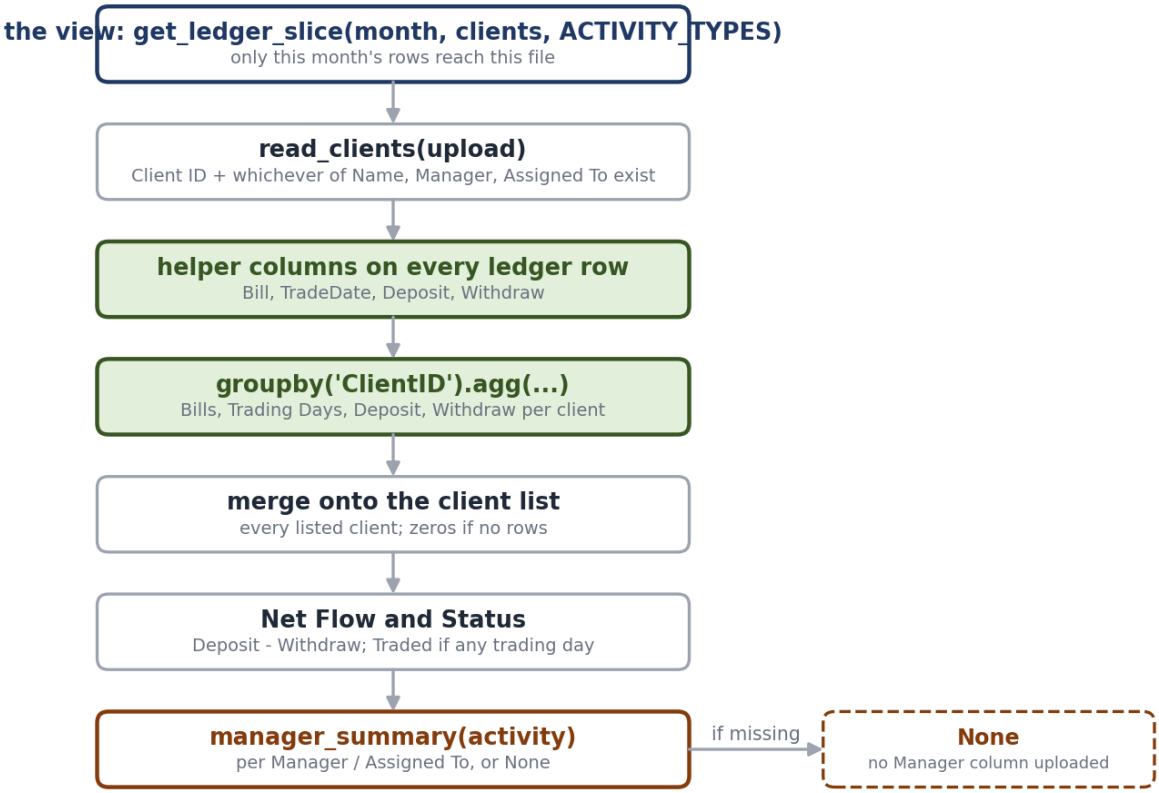
Facts that shape the design

Fact	So the code...	Piece
Uploaded lists differ: some have Name, Manager, Assigned To; some don't	keeps what it finds	`[c for c in OPTIONAL_COLUMNS if c in df.columns]`
A client may appear twice in the list	keeps one row per ID	`drop_duplicates('Client ID')`
Several bills on one day are one trading day	counts distinct dates	`dt.normalize() + nunique`
A cancelled withdrawal gives money back	subtracts it from withdrawals	`Withdraw = W - WITHDRAW CANCELLED`
Listed clients may have no rows at all	still reports them, with zeros	`left merge + fillna`
Without a Manager column there's nothing to summarise	returns None	`if 'Manager' not in columns`

The whole file as a mindmap



Part 2. The big picture, with a worked example



A worked example

ClientID	CreatedAt	LedgerType	Debit	Credit
A	2026-08-02 09:00	BILL	10	0
A	2026-08-02 15:00	BILL	0	5
A	2026-08-05	DEPOSIT	0	1000
A	2026-08-06	WITHDRAW	300	0
A	2026-08-07	WITHDRAW CANCELLED	0	100
B	2026-08-09	DEPOSIT	0	(blank)

Client list: A (Manager m1, Assigned To x), B (m1, y), C (no manager, no ledger rows).

Client	Bills	Trading Days	Deposit	Withdraw	Net Flow	Status
A	2	1 (both bills on 2 Aug)	1000	200 (300 - 100)	800	Traded
B	0	0	0 (blank credit)	0	0	Not Traded
C	0	0	0	0	0	Not Traded

Manager	Assigned To	Total Clients	Traded	Traded %
m1	x	1	1	100.0
m1	y	1	0	0.0
Unassigned	Unassigned	1	0	0.0

Headline: 3 clients, 1 traded, 1 trading day, net flow 800.0. B deposited, but a deposit isn't a trade, so B is Not Traded.

Part 3. Build it step by step

Step 1. read_clients: keep what was uploaded

What I was thinking
"Only Client ID is required. The others are useful when present, so build the list of columns to keep from what the file actually has."
"Exports sometimes repeat a client; one row per ID keeps counts honest."
"reset_index(drop=True) gives a clean 0, 1, 2... index after rows were dropped."

```
ACTIVITY_TYPES = ["BILL", "DEPOSIT", "WITHDRAW", "WITHDRAW CANCELLED"]
OPTIONAL_COLUMNS = ["Name", "Manager", "Assigned To"]


def read_clients(file) -> pd.DataFrame:
    """The client list: 'Client ID' is required; Name, Manager and Assigned To are kept when present."""
    df = read_csv(file, required=["Client ID"], text_columns=["Client ID"] + OPTIONAL_COLUMNS)
    keep = ["Client ID"] + [c for c in OPTIONAL_COLUMNS if c in df.columns]
    return df[keep].dropna(subset=["Client ID"]).drop_duplicates("Client ID").reset_index(drop=True)
```

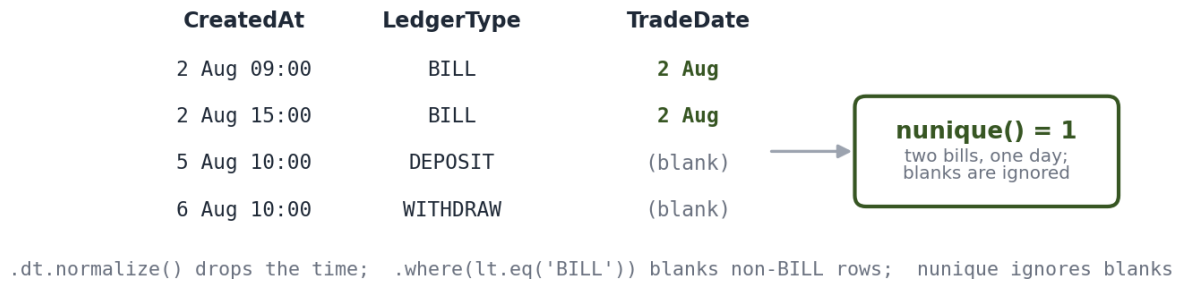
Tool	What it does	Why here
`[c for c in X if c in df.columns]`	A list comprehension with a filter	Only optional columns that exist
`['Client ID'] + [...]`	Joins two lists	Client ID always first
`dropna(subset=['Client ID'])`	Drops rows with no ID	Can't match a blank ID
`drop_duplicates('Client ID')`	Keeps the first row per ID	No double counting
`reset_index(drop=True)`	A fresh 0..n-1 index	Clean for the merge and the Excel

Check yourself
A file with Client ID and Manager (no Name) returns exactly those two columns.

Step 2. Helper columns on every ledger row

What I was thinking
"Same trick as the commission file: make each measure a column, then one groupby."
"Bill is True/False; summing it counts bills. TradeDate is the date only, and blank for non-BILL rows, so it can't count a deposit day as a trading day."
"Withdrawals net of cancellations: each row contributes +debit, -credit or 0."

```
def client_activity(led: pd.DataFrame, clients: pd.DataFrame) -> pd.DataFrame:
    lt = led["LedgerType"].astype(str).str.strip().str.upper()
    debit, credit = led["Debit"].fillna(0), led["Credit"].fillna(0)
    per_client = (led
        .assign(Bill=lt.eq("BILL"),
                TradeDate=pd.to_datetime(led["CreatedAt"]).dt.normalize().where(lt.eq("BILL")),
                Deposit=credit.where(lt.eq("DEPOSIT"), 0),
                Withdraw=debit.where(lt.eq("WITHDRAW"), 0) - credit.where(lt.eq("WITHDRAW CANCELLED"), 0))
```



Tool	What it does	Why here
`lt.eq('BILL')`	True on BILL rows	Summing True values counts them
`.dt.normalize()`	Midnight of the same day	Two bills on one day share a date
`.where(lt.eq('BILL'))`	Blank (NaT) where not a bill	Deposit days don't count as trading days
`credit.where(cond, 0)`	The amount on matching rows, 0 elsewhere	Deposits
`a - b on two Series`	Row-by-row subtraction	Withdrawals minus cancellations

Check yourself
For client A's rows: Bill is True twice, TradeDate is 2 Aug twice and blank three times.

Step 3. One row per client

What I was thinking
"Named aggregation gives each output column a clear name and a clear source."
"Here I keep ClientID as the index (no as_index=False), because the next step merges on the index."

```
        .groupby("ClientID")
        .agg(**{"Bills": ("Bill", "sum"), "Trading Days": ("TradeDate", "nunique"),
                "Deposit": ("Deposit", "sum"), "Withdraw": ("Withdraw", "sum")}))
```

Tool	What it does	Why here
`groupby('ClientID')`	One group per client	ClientID becomes the index
`('Bill', 'sum')`	Counts True values	Number of bills
`('TradeDate', 'nunique')`	Distinct non-blank dates	Trading days
`.agg(**{...})`	Names with spaces	'Trading Days' isn't a valid keyword name

Check yourself
per_client for A: Bills 2, Trading Days 1, Deposit 1000, Withdraw 200.

Step 4. Back onto the client list, then the result columns

What I was thinking
"The uploaded list decides who appears. A left merge from it keeps clients with no ledger rows."
"per_client has ClientID in its index, so right_index=True matches 'Client ID' against that index."
"After a left merge, missing counts are NaN floats; fill them and make them whole numbers again."

```
    out = clients.merge(per_client, left_on="Client ID", right_index=True, how="left")
    out[["Bills", "Trading Days"]] = out[["Bills", "Trading Days"]].fillna(0).astype(int)
    out[["Deposit", "Withdraw"]] = out[["Deposit", "Withdraw"]].fillna(0.0)
    out["Net Flow"] = out["Deposit"] - out["Withdraw"]
    out["Status"] = np.where(out["Trading Days"] > 0, "Traded", "Not Traded")
    return out
```

Tool	What it does	Why here
`merge(left_on=..., right_index=True)`	Column on the left, index on the right	No reset_index needed
`how='left'`	Every row of the client list stays	C appears with zeros
`.fillna(0).astype(int)`	NaN to 0, then int	Counts print as 0, not 0.0
`np.where(days > 0, 'Traded', 'Not Traded')`	Two labels	Traded means at least one trading day

Check yourself
Client C appears with Bills 0, Trading Days 0, Status 'Not Traded'.

Step 5. manager_summary: only when it makes sense

What I was thinking
"No Manager column means no summary: return None, and let the Excel and the page skip that part."
"Group by Manager, and also by Assigned To if it exists: build the key list from the columns."
"Blank managers would vanish from groupby, so label them 'Unassigned' first. A dict comprehension fills each key column in one assign."

```
def manager_summary(activity: pd.DataFrame):
    if "Manager" not in activity.columns:
        return None
    keys = ["Manager"] + (["Assigned To"] if "Assigned To" in activity.columns else [])
    a = activity.assign(**{k: activity[k].fillna("Unassigned") for k in keys},
                        traded=activity["Status"].eq("Traded"))
    out = a.groupby(keys, as_index=False).agg(**{"Total Clients": ("Client ID", "nunique"),
                                                 "Traded": ("traded", "sum"),
                                                 "Trading Days": ("Trading Days", "sum"),
                                                 "Net Flow": ("Net Flow", "sum")})
    out["Traded %"] = (out["Traded"] / out["Total Clients"] * 100).round(1)
    return out
```

Tool	What it does	Why here
`return None`	An explicit 'nothing'	Callers check 'is not None'
`[...] + ([...] if cond else [])`	Adds a key only when it exists	Works with or without Assigned To
`{k: ... for k in keys}`	A dict comprehension	One fillna per key column
`assign(**dict, traded=...)`	Unpacks the dict into keyword arguments	Several columns in one call
`groupby(keys, as_index=False)`	Groups by one or two columns	Keys stay as columns
`(Traded / Total * 100).round(1)`	Percentage, one decimal	Computed after the totals

Check yourself
Without a Manager column the result is None; with it, the worked example gives three groups.

Step 6. build_activity: one call for the view

What I was thinking
"The view needs three things: the client table, the summary (or None), and four numbers to save on the run."

```
def build_activity(led: pd.DataFrame, clients: pd.DataFrame) -> dict:
    activity = client_activity(led, clients)
    return {
        "clients": activity,
        "summary": manager_summary(activity),
        "headline": {
            "clients": int(len(activity)),
            "traded": int(activity["Status"].eq("Traded").sum()),
            "trading_days": int(activity["Trading Days"].sum()),
            "net_flow": round(float(activity["Net Flow"].sum()), 2),
        },
    }
```

Tool	What it does	Why here
`int(len(...))`	Number of rows	Clients in the report
`.eq('Traded').sum()`	Counts True	Traded clients
`round(float(x), 2)`	A plain float, two decimals	Saved into a DecimalField by the view

Check yourself
The worked example's headline is {'clients': 3, 'traded': 1, 'trading_days': 1, 'net_flow': 800.0}.

Part 4. Mistakes this file prevents

If you wrote...	What would happen	What the code does instead
`df[['Client ID', 'Name', 'Manager']]`	KeyError when an upload lacks Name	Keep only optional columns that exist
`groupby(...).size() for trading days`	Counts bills, not days	TradeDate + nunique
`dt.normalize() on every row`	Deposit days counted as trading days	.where(lt.eq('BILL')) first
`Withdraw = debit on WITHDRAW only`	Cancelled withdrawals counted as money out	Subtract WITHDRAW CANCELLED credit
`per_client.merge(clients)`	Clients without rows disappear	clients.merge(..., how='left')
`groupby('Manager') with blanks`	Clients without a manager vanish	fillna('Unassigned') first
`if summary: in the caller`	'truth value of a DataFrame is ambiguous'	Callers use 'is not None'
`Calling it with the full ledger`	All-time activity labelled as one month	The view passes get_ledger_slice(...)

Part 5. Checkpoints

Extracted from this guide and run against the project; they pass.

```
from io import BytesIO

import pandas as pd
from django.test import SimpleTestCase

from activity import services


class ActivityCheckpoints(SimpleTestCase):
    def setUp(self):
        self.led = pd.DataFrame({
            "ClientID": ["A", "A", "A", "A", "A", "B"],
            "CreatedAt": pd.to_datetime(["2026-08-02 09:00", "2026-08-02 15:00", "2026-08-05 10:00",
                                         "2026-08-06 10:00", "2026-08-07 10:00", "2026-08-09 10:00"]),
            "LedgerType": ["BILL", "BILL", "DEPOSIT", "WITHDRAW", "WITHDRAW CANCELLED", "DEPOSIT"],
            "Debit": [10.0, 0.0, 0.0, 300.0, 0.0, 0.0],
            "Credit": [0.0, 5.0, 1000.0, 0.0, 100.0, None]})
        self.clients = pd.DataFrame({"Client ID": ["A", "B", "C"], "Manager": ["m1", "m1", None],
                                     "Assigned To": ["x", "y", None]})

    def test_worked_example(self):
        r = services.build_activity(self.led, self.clients)
        a = r["clients"].set_index("Client ID")
        self.assertEqual((a.loc["A", "Bills"], a.loc["A", "Trading Days"], a.loc["A", "Withdraw"]), (2, 1, 200))
        self.assertEqual(a.loc["B", "Status"], "Not Traded")
        self.assertEqual(r["headline"], {"clients": 3, "traded": 1, "trading_days": 1, "net_flow": 800.0})

    def test_summary_groups(self):
        s = services.build_activity(self.led, self.clients)["summary"].set_index(["Manager", "Assigned To"])
        self.assertEqual(s.loc[("m1", "x"), "Traded %"], 100.0)
        self.assertEqual(s.loc[("Unassigned", "Unassigned"), "Total Clients"], 1)

    def test_no_manager_no_summary(self):
        self.assertIsNone(services.build_activity(self.led, self.clients[["Client ID"]])["summary"])

    def test_read_clients_keeps_what_exists(self):
        csv = b"Client ID,Manager,Other\nA,m1,z\nA,m1,z\n,m2,z\n"
        df = services.read_clients(BytesIO(csv))
        self.assertEqual(list(df.columns), ["Client ID", "Manager"])
        self.assertEqual(df["Client ID"].tolist(), ["A"])
```

Exercise: a safer contract

Add an optional `month=None` parameter to `build_activity`. When given, filter `led` with `core.periods.month_range` before anything else. Write the test first: a row on 1 Sep must not count for '2026-08'.

Explain it back

Answer each in one or two sentences, without looking. The answers follow.

1. Why does read_clients build its column list instead of naming all four columns?
2. Why is TradeDate blank on non-BILL rows?
3. Why does summing the Bill column count bills?
4. Why merge with right_index=True here?
5. Why are Bills and Trading Days turned back into int after the merge?
6. Why does manager_summary return None, and how must the caller test for it?
7. Who decides which month this report covers?

Answers

1. Uploads differ; naming columns that might not exist would raise KeyError.
2. So nunique counts only days with a bill; deposits and withdrawals don't make a trading day.
3. True counts as 1 and False as 0.
4. per_client keeps ClientID as its index after groupby, so the merge matches the list's column to that index.
5. A left merge leaves NaN for clients without rows, which makes the columns float; fillna(0).astype(int) restores counts.
6. There is nothing to summarise without a Manager column. Test with 'is not None', never 'if summary'.
7. The caller: the view passes only that month's rows through get_ledger_slice.

Appendix. The file, with line numbers

activity/services.py

```
  1  """Client trading activity for one month. Plain pandas: no Django imports."""
  2  import numpy as np
  3  import pandas as pd
  4
  5  from core.readers import read_csv
  6
  7  ACTIVITY_TYPES = ["BILL", "DEPOSIT", "WITHDRAW", "WITHDRAW CANCELLED"]
  8  OPTIONAL_COLUMNS = ["Name", "Manager", "Assigned To"]
  9
 10
 11  def read_clients(file) -> pd.DataFrame:
 12      """The client list: 'Client ID' is required; Name, Manager and Assigned To are kept when present."""
 13      df = read_csv(file, required=["Client ID"], text_columns=["Client ID"] + OPTIONAL_COLUMNS)
 14      keep = ["Client ID"] + [c for c in OPTIONAL_COLUMNS if c in df.columns]
 15      return df[keep].dropna(subset=["Client ID"]).drop_duplicates("Client ID").reset_index(drop=True)
 16
 17
 18  def client_activity(led: pd.DataFrame, clients: pd.DataFrame) -> pd.DataFrame:
 19      """One row per listed client: bills, trading days, deposits, net withdrawals, traded or not."""
 20      lt = led["LedgerType"].astype(str).str.strip().str.upper()
 21      debit, credit = led["Debit"].fillna(0), led["Credit"].fillna(0)
 22      per_client = (led
 23          .assign(Bill=lt.eq("BILL"),
 24                  TradeDate=pd.to_datetime(led["CreatedAt"]).dt.normalize().where(lt.eq("BILL")),
 25                  Deposit=credit.where(lt.eq("DEPOSIT"), 0),
 26                  Withdraw=debit.where(lt.eq("WITHDRAW"), 0) - credit.where(lt.eq("WITHDRAW CANCELLED"), 0))
 27          .groupby("ClientID")
 28          .agg(**{"Bills": ("Bill", "sum"), "Trading Days": ("TradeDate", "nunique"),
 29                  "Deposit": ("Deposit", "sum"), "Withdraw": ("Withdraw", "sum")}))
 30
 31      out = clients.merge(per_client, left_on="Client ID", right_index=True, how="left")
 32      out[["Bills", "Trading Days"]] = out[["Bills", "Trading Days"]].fillna(0).astype(int)
 33      out[["Deposit", "Withdraw"]] = out[["Deposit", "Withdraw"]].fillna(0.0)
 34      out["Net Flow"] = out["Deposit"] - out["Withdraw"]
 35      out["Status"] = np.where(out["Trading Days"] > 0, "Traded", "Not Traded")
 36      return out
 37
 38
 39  def manager_summary(activity: pd.DataFrame):
 40      """Clients and traded clients per Manager (and Assigned To); None when there's no Manager column."""
 41      if "Manager" not in activity.columns:
 42          return None
 43      keys = ["Manager"] + (["Assigned To"] if "Assigned To" in activity.columns else [])
 44      a = activity.assign(**{k: activity[k].fillna("Unassigned") for k in keys},
 45                          traded=activity["Status"].eq("Traded"))
 46      out = a.groupby(keys, as_index=False).agg(**{"Total Clients": ("Client ID", "nunique"),
 47                                                   "Traded": ("traded", "sum"),
 48                                                   "Trading Days": ("Trading Days", "sum"),
 49                                                   "Net Flow": ("Net Flow", "sum")})
 50      out["Traded %"] = (out["Traded"] / out["Total Clients"] * 100).round(1)
 51      return out
 52
 53
 54  def build_activity(led: pd.DataFrame, clients: pd.DataFrame) -> dict:
 55      activity = client_activity(led, clients)
 56      return {
 57          "clients": activity,
 58          "summary": manager_summary(activity),
 59          "headline": {
 60              "clients": int(len(activity)),
 61              "traded": int(activity["Status"].eq("Traded").sum()),
 62              "trading_days": int(activity["Trading Days"].sum()),
 63              "net_flow": round(float(activity["Net Flow"].sum()), 2),
 64          },
 65      }
```

Lines	Block	Purpose
2-8	`Imports + constants`	Types and optional columns
11-15	`read_clients()`	Keep what was uploaded
18-29	`client_activity(): per client`	Helper columns + groupby
31-36	`client_activity(): result`	Merge, fill, Net Flow, Status
39-51	`manager_summary()`	Per manager and agent, or None
54-65	`build_activity()`	One call for the view